Numerov Method for Bound States in the One-Dimensional Schrödinger Equation - Project Outline and Motivation -

Alon Sportes (ID: 204498745)

Contents

Table of Contents 1

List of Figures 3

1 What problem are we solving? 4

2 Which course topics are used? 4
2.1 Lecture 1: Numerical errors and computational project expectations . . . 4
2.2 Lecture 3: Numerical differentiation and integration . . . 5
2.3 Lecture 4: Root finding . . . 6
2.4 Lecture 5: ODEs, boundary-value problems, eigenvalue problems, and shooting . . . 7
2.5 Lecture 6: PDEs and Schrödinger equation context . . . 8
2.6 What is not used . . . 8

3 The Numerov algorithm 8
3.1 In-depth explanation of the Numerov algorithm . . . 8
3.2 Why Numerov? . . . 9
3.3 How the code uses the Numerov algorithm . . . 10

4 Why shooting? 10

5 Why parity? 11

6 Why root finding and bisection? 12

7 What each code file does 13
7.1 `src/potentials.py` . . . 13
7.2 `src/numerov.py` . . . 14
7.3 `src/shooting.py` . . . 14
7.4 `src/analysis.py` . . . 15
7.5 `src/rk4_compare.py` . . . 15
7.6 `src/scattering.py` . . . 16
7.7 `src/plotting.py` . . . 16
7.8 `src/experiments.py` . . . 17
7.9 `scripts/run_solver.py` . . . 17
7.10 `tests/test_solver.py` . . . 18

8 What the project demonstrates **18**
8.1 Level 1: Exact benchmarks 18
8.2 Level 2: Convergence 19
8.3 Level 3: New physics 19

9 What is apparent from the lectures and what is beyond them? **19**
9.1 Clearly from the lectures 19
9.2 Partly from the lectures, but specialized to quantum mechanics 20
9.3 Beyond the slides 20
9.4 Not used from the lectures 21

10 The clean explanation for a reviewer **21**

List of Figures

1 What problem are we solving?

The project solves the one-dimensional time-independent Schrödinger equation. In the dimensionless convention (where $\hbar = 1$, $m = 1$, and E is measured in units of the potential depth) used in the code, it is written as

$$\psi''(x) = 2\,[V(x) - E]\psi(x). \tag{1}$$

The unknowns are:

- $\psi(x)$: the wavefunction,
- E: the energy eigenvalue,
- $V(x)$: the chosen potential.

This is not just an ordinary differential equation problem. It is a boundary-value eigenvalue problem. Only special values of E give physically valid bound states. This matches the proposal and the report structure exactly.

2 Which course topics are used?

2.1 Lecture 1: Numerical errors and computational project expectations

This project uses the main idea from Lecture 1: numerical computation is approximate, so numerical errors must be controlled and the method must be documented. The final project instructions also require code documentation and a short writeup explaining the problem, the method, the results, and the convergence and correctness checks.

In this project, this appears through:

- Convergence studies: implemented in `src/analysis.py` (e.g. `convergence_vs_grid()`, `convergence_vs_box` `convergence_vs_grid_successive()`) and executed in `src/experiments.py`.
- Comparison with exact solutions: implemented in `src/analysis.py` (`exact_square_well_energies()`, `exact_harmonic_oscillator_energies()`) and used in the square well and harmonic oscillator experiments.
- Normalization checks: implemented in `src/numerov.py` via `normalize_wavefunction()` and verified in `tests/test_solver.py`.
- Automated tests: implemented in `tests/test_solver.py`, covering correctness, convergence, and regression behavior.
- Root-finding diagnostics: orchestrated in `src/diagnostics.py`, which builds both global mismatch scans and zoomed per-root bisection views for the bound-state experiments.
- README documentation: provided in `README.md`, describing usage, structure, and project goals.
- Discussion of truncation error, round-off error, and numerical floors: reflected in `src/analysis.py` (convergence diagnostics) and discussed in the report through convergence plots and slope interpretation.

A very important example is the infinite square well convergence study. The solver originally produced very accurate energies, but the observed convergence order was not ideal. The issue was not the Numerov recurrence itself, but the startup values used to seed the two-step recurrence. After adding higher Taylor terms to the parity-based startup conditions, the square-well convergence slopes improved to approximately fourth order. A similar lesson appeared later in the quartic double-well

study: the convergence behavior was polluted until the potential offset, startup expansion, and convergence methodology were made consistent. These are exactly the kinds of numerical-error diagnoses emphasized in the course.

2.2 Lecture 3: Numerical differentiation and integration

Numerical integration is used when normalizing the wavefunction:

$$\|\psi\| = \sqrt{\int |\psi(x)|^2 \, dx}. \tag{2}$$

In the code, this is done with `np.trapezoid`. The reason is physical: quantum wavefunctions must satisfy

$$\int |\psi(x)|^2 \, dx = 1. \tag{3}$$

Therefore, normalization is not optional. It makes the solution physically meaningful.

The trapezoidal rule is used for this integration rather than more elaborate quadrature methods. In general, higher-order integration rules can be more accurate when the integrand is smooth and the grid satisfies the assumptions of the method. However, the best numerical method is not always the highest-order one; it should match the role the calculation plays in the algorithm. Here, normalization is only a post-processing step. It rescales a wavefunction that has already been computed by the ODE solver, and it is not the step that determines the eigenvalue. The main sources of error in the project come from ODE discretization, boundary conditions, domain truncation, and root-finding accuracy, not from the final normalization integral.

This is why the trapezoidal rule is sufficient and appropriate here. The wavefunctions are already sampled on the same uniform grid used by the Numerov solver, and they are smooth and well resolved in the accepted solutions. A more complicated integration rule would not materially change the physical conclusions, such as the eigenvalues, node structure, tunneling splitting, or scattering behavior. It would mainly add extra assumptions and implementation details.

Simpson's rule is a good example. It is higher order than the trapezoidal rule for smooth functions, so it would also be a valid choice for normalization in many cases. But Simpson's rule usually requires a compatible number of intervals and a uniform grid structure that fits its stencil. Since the project changes grid sizes in convergence studies and uses the integral only to normalize already-computed states, Simpson's rule would add bookkeeping constraints without improving the dominant numerical accuracy. Therefore, the trapezoidal rule is chosen because it is simple, robust, grid-flexible, and accurate enough for the purpose it serves in this project.

Numerical differentiation is also used in `derivative_at_right_edge()`. This is especially important for inward shooting. For even states, the condition at the origin is

$$\psi'(0) = 0. \tag{4}$$

If the derivative is computed with a low-order stencil, then the whole eigenvalue calculation can lose accuracy even if the Numerov recurrence itself is high order. This happened in the harmonic oscillator case. The code now uses a fourth-order backward stencil when enough points are available. This keeps the boundary-condition accuracy consistent with the rest of the method.

The function `derivative_at_right_edge()` uses a backward finite-difference stencil to estimate the derivative at the boundary. This choice is dictated by the location of the evaluation point: at the right edge of the grid there are no points available to the right, so a forward or central difference would require values outside the computational domain. A backward stencil uses only existing grid points $x_N, x_{N-1}, x_{N-2}, \ldots$ and is therefore the natural choice.

When enough grid points are available, the implementation uses a fourth-order backward stencil. This is important because the overall solver is high order due to the Numerov recurrence. If the derivative used to enforce the boundary condition were only second order, it would become the dominant error and reduce the accuracy of the eigenvalue calculation. The high-order backward stencil ensures that the derivative-based boundary check, such as $\psi'(0) = 0$ for even states in inward shooting, remains consistent with the accuracy of the integration method.

To compute this derivative explicitly, the fourth-order backward stencil has the form

$$\psi'(x_N) \approx \frac{25\psi_N - 48\psi_{N-1} + 36\psi_{N-2} - 16\psi_{N-3} + 3\psi_{N-4}}{12h}. \tag{5}$$

This expression uses only grid points inside the domain and achieves $O(h^4)$ accuracy. Including this higher-order stencil is essential so that the boundary-derivative evaluation does not reduce the overall convergence order of the Numerov method.

2.3 Lecture 4: Root finding

Root finding is used to determine the allowed energies. The project uses bisection: first detect a sign change, then repeatedly cut the interval in half.

In the code this appears in:

- `find_brackets_outward_shooting()`: scans a chosen energy interval, evaluates the outward-shooting mismatch function, and identifies intervals where the mismatch changes sign. These intervals are candidate eigenvalue brackets.
- `bisect_energy_outward_shooting()`: takes one outward-shooting bracket and repeatedly bisects it until the energy root is isolated accurately. In the latest implementation, it also applies a safeguarded secant-style polishing step to improve the final estimate.
- `find_brackets_inward_shooting()`: performs the same bracketing idea for inward shooting, where the mismatch is the parity residual at the origin after integrating inward from the decaying tail.
- `bisect_energy_inward_shooting()`: refines an inward-shooting energy bracket by bisection until the even or odd parity condition at the origin is satisfied to the required tolerance.
- `RK4_find_brackets()`: scans energies for the RK4 harmonic-oscillator comparison and finds sign-changing brackets of the RK4 mismatch function.
- `RK4_bisect_energy()`: refines an RK4 bracket by bisection to find the harmonic-oscillator eigenvalue computed by the RK4 shooting formulation.

Bisection is used because it is stable and easy to justify. Newton's method is faster, but it requires derivatives and can fail if the starting guess is poor. Bisection is slower, but it is robust. In the latest solver, the bisection routine was also adjusted so it does not stop too early only because the energy bracket became small; it now uses a safeguarded secant polishing step to improve the final root estimate.

The project also includes root-finding diagnostics. These are now organized in `src/diagnostics.py`. They are not strictly required for computing eigenvalues, but they are useful because they show both the full mismatch curve and zoomed views of individual sign-changing brackets, making the bisection process visible rather than hidden.

2.4 Lecture 5: ODEs, boundary-value problems, eigenvalue problems, and shooting

This is the core lecture for the project. The Schrödinger equation is a second-order ODE, but unlike an initial-value problem, the correct energy is not known in advance. Therefore, the method is:

1. guess an energy E,
2. integrate the ODE,
3. check the boundary condition,
4. improve the energy guess.

That is the shooting method.

The project uses two shooting strategies:

- outward shooting from $x = 0$,
- inward shooting from $x_{\max}$ to $x = 0$.

Here $x_{\max}$ denotes the finite boundary used to approximate an infinite or sufficiently large spatial domain. In practice, the continuous problem is truncated to the interval $[0, x_{\max}]$ (or $[-x_{\max}, x_{\max}]$ before using parity). The value of $x_{\max}$ must be chosen large enough so that the wavefunction has effectively decayed to zero at the boundary for bound states. The sensitivity of the results to this choice is studied explicitly through convergence with respect to domain size.

Outward shooting is used for many symmetric finite-domain or effectively finite-domain problems. Inward shooting is used mainly for the harmonic oscillator and the RK4 comparison. This is because the harmonic oscillator is defined on an infinite domain, and outward integration can pick up the unphysical growing exponential tail. Inward shooting starts from the decaying asymptotic side and imposes the parity condition at the origin.

Inward shooting is therefore not universally better than outward shooting. It is better only when the physical boundary condition is easiest and most stable to impose at large distance. This is true for confining potentials with exponentially decaying bound states, such as the harmonic oscillator. For finite-domain or symmetric bound-state problems, outward shooting from the origin is usually more natural because the parity conditions at $x = 0$ are exact and do not require approximating an asymptotic tail. For scattering problems, the physical boundary conditions are incoming, reflected, and transmitted waves rather than exponential decay, so inward bound-state shooting is not the correct formulation. The general rule is that the shooting direction should be chosen according to where the physical boundary condition is clearest and most numerically stable.

This does not introduce an inconsistency in the numerical method. The overall framework remains the same for all potentials: the Schrödinger equation is integrated numerically, a boundary mismatch function $F(E)$ is defined, and the eigenvalues are obtained by solving $F(E) = 0$ using root finding. What changes between inward and outward shooting is not the method itself, but the location at which the boundary condition is imposed.

This choice does not rely on the existence of an analytic solution. Instead, it follows from qualitative properties of the potential, such as symmetry, finite or infinite domain, and asymptotic behavior. For symmetric finite-domain or effectively finite-domain problems, outward shooting is natural because the parity conditions at $x = 0$ are exact. For confining potentials on an infinite domain, inward shooting is more stable because the physically correct solution decays at large x. The numerical strategy is therefore consistent across all cases, while still adapting to the physics of each potential to ensure stability and accuracy.

2.5 Lecture 6: PDEs and Schrödinger equation context

Lecture 6 mentions the Schrödinger equation as one of the important PDEs in physics. This project uses the time-independent Schrödinger equation in one dimension, which becomes an ODE eigenvalue problem. Therefore, this project is not solving a time-dependent PDE. It is using an ODE method on the stationary quantum problem.

2.6 What is not used

The project does not use:

- finite-volume methods,
- upwind methods,
- CFL conditions,
- Riemann solvers,
- shock capturing,
- slope limiters,
- Euler fluid equations,
- MHD,
- Particle in Cell.

Those topics belong to the advection, hydrodynamics, Burgers equation, Euler equations, and PIC parts of the course. They are not part of this project.

3 The Numerov algorithm

3.1 In-depth explanation of the Numerov algorithm

After nondimensionalization, the one-dimensional time-independent Schrödinger equation is written as

$$\psi''(x) = q(x)\psi(x), \tag{6}$$

where

$$q(x) = 2[V(x) - E]. \tag{7}$$

This is a linear second-order ordinary differential equation with no first-derivative term, which is exactly the structure for which the Numerov algorithm is designed.

The main idea is to discretize space on a uniform grid $x_n = x_0 + nh$ and derive a recurrence relation for the wavefunction values $\psi_n = \psi(x_n)$. Instead of replacing ψ'' with the simplest second-difference approximation, Numerov combines Taylor expansions about neighboring points in a way that cancels lower-order truncation errors. The resulting method advances the solution using ψ_{n-1} and ψ_n to obtain ψ_{n+1} while incorporating the local values of $q(x)$ at the same three grid points.

For an equation of the form $\psi''(x) = q(x)\psi(x)$, the standard Numerov update can be written as

$$\left(1 - \frac{h^2}{12}q_{n+1}\right)\psi_{n+1} = 2\left(1 + \frac{5h^2}{12}q_n\right)\psi_n - \left(1 - \frac{h^2}{12}q_{n-1}\right)\psi_{n-1}, \tag{8}$$

where $q_n = q(x_n)$. Solving for ψ_{n+1} gives an explicit two-step marching scheme. Its local truncation error is higher order than that of basic finite-difference stepping, so in practice it is much more accurate

than Euler-type methods on the same grid. For smooth problems, this is the reason it is a standard tool for Schrödinger eigenvalue calculations.

The high accuracy comes from exploiting the special structure of the equation rather than treating it as a completely general first-order system. A generic Runge-Kutta integrator can also solve the problem, but it introduces an auxiliary variable for ψ' and does not take direct advantage of the fact that the physical equation is already second order and linear. Numerov is therefore both specialized and efficient: one stores only the wavefunction sequence and the coefficient function $q(x)$, yet still obtains high-order propagation.

There is, however, an important practical consequence of the recurrence: Numerov is a two-step method. It cannot start from a single boundary value alone. It needs two nearby values, typically ψ_0 and ψ_1, before the recurrence can be applied. That means the startup formula is part of the algorithm, not just a minor implementation detail. If the initialization is too crude, then the formally high-order recurrence can be contaminated by low-order starting error. This project encountered exactly that issue during convergence studies.

For symmetric potentials, the startup values are obtained from parity and a Taylor expansion about the origin. For even states,

$$\psi(0) = 1, \qquad \psi'(0) = 0, \tag{9}$$

while for odd states,

$$\psi(0) = 0, \qquad \psi'(0) = 1. \tag{10}$$

These conditions determine the local series expansion and therefore the first usable grid values. The latest implementation keeps higher Taylor terms so that the first Numerov step remains consistent with the overall order of the method:

- even states include the h^4 term,
- odd states include the h^5 term.

The quartic double-well problem required one more refinement. Near the origin, the local curvature of $q(x)$ affects the startup series, so the code was extended to include the relevant $q''(0)$ terms for both parities. Without this correction, one of the low-lying double-well states showed a reduced convergence slope even though the recurrence itself was correct. After the startup was upgraded, the state recovered high-order behavior consistent with Numerov. This is a useful lesson for the project: the algorithm is not just the recurrence formula, but also the consistent initialization and boundary diagnostics that allow its formal accuracy to appear in actual computations.

3.2 Why Numerov?

Numerov is the natural method here because the Schrödinger equation already has the exact structure it expects: a second-order ODE with no first-derivative term. That makes it more appropriate than a generic ODE solver if the goal is to compute bound-state wavefunctions and eigenvalues efficiently on a uniform grid.

It is also a good match to the goals of the project. I want a method that is accurate enough to support convergence studies, simple enough to implement from first principles, and specialized enough that numerical issues can be interpreted physically and algorithmically. Numerov satisfies all three. It is compact, mathematically transparent, and widely used in textbook Schrödinger problems, including the treatment in Pang based on wavefunction integration plus eigenvalue search.

The project results also justify the choice. Once the startup formulas and boundary derivative estimates were made consistent with the method's order, the solver produced clean convergence for the infinite square well, harmonic oscillator, finite square well, and quartic double well. That would be harder to diagnose pedagogically with a more black-box integrator. Numerov therefore serves not only as an

effective solver, but also as a useful framework for studying numerical error, parity reduction, domain truncation, and tunneling-sensitive states.

3.3 How the code uses the Numerov algorithm

In the codebase, Numerov is the propagation engine inside the larger shooting-based eigenvalue solver. The functions in `src/numerov.py` implement the method-specific operations:

- `q_from_energy()` builds the coefficient function $q(x)$ from the potential and trial energy,
- `numerov_outward()` performs the Numerov recurrence across the grid,
- `normalize_wavefunction()` rescales the final wavefunction so that total probability is 1,
- `derivative_at_right_edge()` estimates the derivative at the boundary when a derivative-based mismatch is needed.

The startup values used by the recurrence are supplied from `src/shooting.py`. In particular, `initial_conditions_` constructs the parity-based Taylor expansions near the origin, and `half_domain_wavefunction_outward_shooting` uses those values to launch the outward integration on the half domain. The shooting routines then evaluate how well the integrated solution satisfies the target boundary behavior for a given trial energy.

The overall workflow is therefore:

1. choose a trial energy E,
2. construct $q(x) = 2[V(x) - E]$ on the grid,
3. generate parity-consistent startup values near the origin,
4. propagate the wavefunction outward with Numerov,
5. measure the boundary mismatch,
6. adjust E until the mismatch vanishes.

For symmetric potentials, only the half domain must be integrated, after which the full wavefunction is reconstructed from parity and normalized. This is one of the main algorithmic advantages used throughout the project. In that sense, the code does not use Numerov in isolation: it combines Numerov propagation, carefully designed startup conditions, symmetry reduction, boundary-condition diagnostics, and root-finding in energy to build the full bound-state solver.

4 Why shooting?

A bound state must behave correctly at both boundaries. However, Numerov integration needs starting values. Therefore, the algorithm is:

1. choose a trial energy E,
2. choose initial conditions at $x = 0$, or from the decaying tail,
3. integrate with Numerov,
4. check whether the wavefunction satisfies the target boundary condition,
5. adjust E.

This is shooting.

In `src/shooting.py`, the outward shooting logic is built from:

- `initial_conditions_outward_shooting()`,
- `half_domain_wavefunction_outward_shooting()`,
- `boundary_mismatch_outward_shooting()`,
- `find_brackets_outward_shooting()`,
- `bisect_energy_outward_shooting()`,
- `solve_state_from_bracket_outward_shooting()`,
- `solve_symmetric_potential_outward_shooting()`.

The important function is `boundary_mismatch_outward_shooting()`. Here, a *mismatch* means the residual error in the boundary condition after integrating the wavefunction for a trial energy. In other words, after the solver marches across the domain, it evaluates how far the computed wavefunction is from satisfying the required boundary behavior at the endpoint. If that residual is not zero, the trial energy is not an eigenvalue. The function `boundary_mismatch_outward_shooting()` is therefore the code routine that computes this residual for a given energy.

The latest solver also changes how the final mismatch is stored. For outward shooting, the code no longer reports the raw, unnormalized boundary amplitude $\psi(x_{\max})$; it stores the normalized boundary leakage after the full wavefunction has been reconstructed and normalized. This makes that residual scale-invariant and physically more meaningful. For inward shooting, however, the stored residual is the final parity mismatch at the origin, because that is the quantity the inward solver actually drives to zero. So the code now treats `StateSolution.mismatch` as a general final diagnostic residual, with an interpretation that depends on the shooting formulation.

The inward shooting logic is built from:

- `initial_conditions_inward_shooting()`,
- `half_domain_wavefunction_inward_shooting()`,
- `boundary_mismatch_inward_shooting()`,
- `find_brackets_inward_shooting()`,
- `bisect_energy_inward_shooting()`,
- `solve_symmetric_potential_inward_shooting()`.

Inward shooting is used for confining potentials such as the harmonic oscillator. The physical solution decays at large x. If we integrate outward from the origin, the numerical solution can pick up the unphysical growing mode. Inward shooting starts in the forbidden region and integrates toward the origin, giving cleaner harmonic oscillator wavefunctions and more reliable parity enforcement.

5 Why parity?

The main bound-state potentials are symmetric:

$$V(-x) = V(x). \tag{11}$$

For symmetric potentials, quantum states are either even or odd.

Even states satisfy

$$\psi(-x) = \psi(x), \tag{12}$$

and odd states satisfy

$$\psi(-x) = -\psi(x). \tag{13}$$

Therefore, instead of integrating from $-x_{\max}$ to $x_{\max}$, the code integrates only from 0 to $x_{\max}$.

In code, `initial_conditions_outward_shooting()` uses:

- even: $\psi(0) = 1$, $\psi'(0) = 0$,
- odd: $\psi(0) = 0$, $\psi'(0) = 1$.

Then `build_full_wavefunction()` reflects the half-domain solution to the left side.

This choice makes the algorithm simpler, faster, and more stable. It also lets the states be classified physically as even or odd.

Parity is also useful for the double well because tunneling splitting appears as pairs of even and odd states. The lowest two states are symmetric and antisymmetric combinations of wavefunctions localized in the two wells.

6 Why root finding and bisection?

For each trial energy, the boundary mismatch gives a number:

$$F(E) = \text{mismatch}(E). \tag{14}$$

The precise definition of the mismatch depends on the shooting direction and on which boundary condition is being enforced.

Outward shooting (from $x = 0$ to $x_{\max}$):

$$F(E) = \psi_E(x_{\max}). \tag{15}$$

For a true bound state, the wavefunction must vanish at the boundary, so the condition is $\psi_E(x_{\max}) = 0$.

In the implementation, this is evaluated after constructing and normalizing the full wavefunction, so the stored mismatch represents the normalized boundary leakage, which is scale-invariant and physically meaningful.

Inward shooting (from $x_{\max}$ to $x = 0$):

For even states:

$$F(E) = \psi'_E(0), \tag{16}$$

For odd states:

$$F(E) = \psi_E(0). \tag{17}$$

In this case, the mismatch measures how well the parity condition at the origin is satisfied after integrating inward from the decaying tail.

In both formulations, the eigenvalue problem is reduced to solving $F(E) = 0$, but the definition of $F(E)$ reflects the boundary where the physical condition is imposed. This $F(E)$ is the *mismatch function*: it takes an energy E as input and returns the boundary-condition residual produced by the shooting calculation. Its sign and magnitude indicate how the trial solution fails to satisfy the boundary condition.

Allowed eigenvalues satisfy

$$F(E) = 0. \tag{18}$$

So the eigenvalue problem becomes a root-finding problem.

The code first scans energies using `find_brackets()`. It looks for sign changes. If $F(E_1)$ and $F(E_2)$ have opposite signs, then a root is likely between them. Then `bisect_energy()` refines the energy until the interval is small enough.

This is exactly the root-finding logic from the course: bracket a root, then reduce the bracket until the required accuracy is reached.

The code also includes:

- `sample_boundary_mismatch()`,
- `bisection_history()`,
- `sample_inward_decay_mismatch()`,
- `bisection_history_inward_decay()`.

These are mainly for diagnostics and plots. They show that the eigenvalues are not magic numbers. They are roots of a clearly defined numerical function.

7 What each code file does

7.1 `src/potentials.py`

This file defines the physical systems:

- `harmonic_oscillator()`,
- `infinite_square_well_numeric()`,
- `finite_square_well()`,
- `quartic_double_well()`,
- `square_barrier()`,
- `double_square_barrier()`.

The purpose is to separate the physics definitions from the numerical solver. This is good design: the solver does not care which potential is chosen. It only needs $V(x)$. That makes the project modular.

The main potentials have clear roles:

- infinite square well: analytic validation,
- harmonic oscillator: analytic validation and inward shooting,
- finite square well: nontrivial finite bound-state system,
- quartic double well: tunneling and energy splitting, with the shifted potential using the analytic minimum $V_{\min} = -b^2/(4a)$,
- single barrier and double barrier: scattering extension.

A recent correction was made in `quartic_double_well()`: the zero shift now uses the analytic minimum

$$V_{\min} = -\frac{b^2}{4a} \tag{19}$$

instead of the sampled grid minimum. This is important because using the sampled minimum makes the potential itself depend slightly on the grid spacing h, which contaminates a grid-convergence study.

For the quartic double-well potential,

$$V(x) = ax^4 - bx^2, \tag{20}$$

the true minimum can be obtained analytically by minimizing $V(x)$, which gives

$$V_{\min} = -\frac{b^2}{4a}. \tag{21}$$

It is therefore preferable to shift the potential using this exact expression rather than a grid-sampled minimum. The shifted potential is defined as

$$V_{\text{shifted}}(x) = ax^4 - bx^2 - V_{\min}, \tag{22}$$

so that the minima lie at zero energy. This choice makes the energy scale cleaner and easier to interpret, and avoids introducing grid-dependent artifacts into convergence studies.

7.2 `src/numerov.py`

This is the numerical integration engine. Its main role is to take an x grid, a potential V, and a trial energy E, and produce a trial wavefunction ψ.

The most important functions are:

- `q_from_energy()`,
- `numerov_outward()`,
- `normalize_wavefunction()`,
- `derivative_at_right_edge()`.

Important details:

- `numerov_outward()` includes dynamic rescaling to avoid overflow when a non-eigenvalue trial solution grows exponentially.
- `normalize_wavefunction()` rescales before squaring to avoid overflow.
- `derivative_at_right_edge()` uses a fourth-order stencil when possible.

7.3 `src/shooting.py`

This is the eigenvalue solver. Its main role is to try energies, integrate with Numerov, check the boundary mismatch, find the correct eigenvalues, and return normalized eigenstates.

The most important functions are:

- `initial_conditions()`,
- `find_brackets()`,
- `bisect_energy()`,
- `solve_symmetric_potential()`,
- `solve_symmetric_potential_inward_decay()`.

Important latest details: `initial_conditions()` now includes higher Taylor terms so the first Numerov step does not reduce the convergence order. For the quartic double well, the startup was also

extended with the $q''(0)$ contribution. This fixed the square-well convergence slope problem and restored the expected high-order convergence for the problematic double-well state. The bisection routine was also strengthened with a safeguarded secant polishing step. In addition, `StateSolution.mismatch` now stores the final solver residual in a formulation-aware way: normalized boundary leakage for outward shooting, and the final origin-parity mismatch for inward shooting.

7.4 `src/analysis.py`

This is the scientific validation layer. It contains:

- `exact_square_well_energies()`,
- `exact_harmonic_oscillator_energies()`,
- `relative_error()`,
- `estimate_convergence_slopes()`,
- `convergence_vs_grid()`,
- `convergence_vs_grid_successive()`,
- `convergence_vs_box_size()`,
- `convergence_vs_box_size_fixed_spacing()`,
- `splitting_vs_parameter()`.

This is what turns the code from "it runs" into "it was verified scientifically."

For cases without analytic solutions, such as the quartic double well, the code now distinguishes between two convergence strategies. The box-size study is compared against a larger-box reference, while the grid-spacing study uses successive refinements, comparing each grid to the next finer one. This avoids pretending that there is an exact spectrum and avoids introducing an artificial reference floor into the h-convergence study.

A related motivation also appears in the harmonic-oscillator and RK4 studies. When the code measures sensitivity to $x_{\max}$, it does not keep `n_grid` fixed. Instead it keeps the spacing h approximately fixed while the box size changes. Otherwise a box-size study would also change the discretization error and mix two different effects into the same curve. This is why the project includes `convergence_vs_box_size_fixed_spacing()` and its RK4 analogue.

7.5 `src/rk4_compare.py`

This is the method-comparison module. It solves the harmonic oscillator using RK4 shooting.

The harmonic oscillator is used because it has exact energies and a smooth potential, so it is a fair comparison.

RK4 works differently from Numerov. Numerov solves the second-order equation directly. RK4 rewrites it as a first-order system:

$$\psi' = \phi, \tag{23}$$

$$\phi' = 2[V(x) - E]\psi. \tag{24}$$

This means RK4 evolves both ψ and ψ' explicitly.

The comparison is useful because it shows the difference between a specialized solver and a general-purpose ODE solver. It also explains why the derivative boundary condition mattered so much. RK4

carries ψ' as a variable, while Numerov needs a finite-difference derivative estimate when enforcing some boundary conditions.

7.6 src/scattering.py

This is the scattering extension. It computes:

- transmission T,
- reflection R,
- conservation check $T + R$.

The method is:

1. assume the potential is zero far left and far right,
2. impose a transmitted right-moving wave on the far right,
3. integrate backward through the potential with complex Numerov,
4. decompose the left-side wave into incident and reflected parts.

The main functions are:

- `numerov_outward_complex()`,
- `integrate_from_right()`,
- `decompose_left_asymptotic()`,
- `solve_scattering()`,
- `sweep_scattering()`,
- `find_transmission_peaks()`.

This is not the core bound-state project, but it shows that the same Numerov framework can be extended to scattering and resonant tunneling.

7.7 src/plotting.py

This file generates report figures:

- `plot_potential_and_states()`,
- `plot_probability_densities()`,
- `plot_energy_comparison()`,
- `plot_error_curve()`,
- `plot_splitting_curve()`,
- `plot_root_finding_diagnostic()`,
- `plot_scattering_coefficients()`,
- `plot_scattering_potential_and_probability()`,
- `plot_numerov_vs_rk4_errors()`.

These plots let the reviewer visually see the states, convergence, tunneling behavior, scattering behavior, and method comparison. Recent plotting updates increased the displayed energy precision from 4 to 6 decimals and changed the root-diagnostic plots to show the raw mismatch on a symmetric logarithmic scale. This makes even and odd root scans easier to distinguish and avoids hiding structure through clipping or overly aggressive scaling.

7.8 `src/experiments.py`

This file connects the solver to the actual project cases. It runs:

- `run_square_well()`,
- `run_harmonic_oscillator()`,
- `run_double_well()`,
- `run_finite_square_well()`,
- `run_scattering()`.

This file is basically the scientific experiment plan in code.

Important latest double-well additions:

- the default double-well domain was increased from $x_{\max} = 3.0$ to $x_{\max} = 4.0$,
- grid-convergence and box-convergence studies are now separated properly,
- the grid-spacing study now uses successive refinements rather than a single fixed reference,
- the box-size study is still compared against a larger-box reference,
- the double-well plot legend now shows the explicit formula,
- root-finding diagnostics are now generated separately for the double well,
- redundant outputs such as the old `3_double_well_splitting.png` were removed,
- the retained splitting plot is `3_double_well_Numerov_splitting_vs_b.png`,
- each experiment now writes into its own subdirectory under `results/`, for example `results/3_double_well_N`

The same convergence-separation logic is also used in the harmonic-oscillator comparison workflow. The harmonic and RK4 box-size studies keep h approximately fixed while varying $x_{\max}$, so the resulting curves mainly measure finite-domain truncation rather than a mixture of truncation and discretization error.

7.9 `scripts/run_solver.py`

This is the entry point. It:

- clears old results,
- runs all experiments,
- runs tests,
- prints a completion message.

It is intentionally lightweight. Most logic lives in `src/`. The latest runner keeps a single canonical results root at `PROJECT_ROOT/results` and each experiment writes its CSV and PNG outputs into its own dedicated subdirectory there.

There is also a small but important software-design motivation here: the experiment modules are imported only inside `main()`. This keeps the runner lightweight to import, allows the test suite to check `run_solver` import behavior without forcing plotting dependencies immediately, and keeps orchestration separate from the scientific computation itself.

7.10 `tests/test_solver.py`

This file checks correctness. It tests:

- normalization,
- derivative stencil accuracy,
- RK4 bracket detection,
- square-well ground-state accuracy,
- square-well convergence order,
- harmonic oscillator energies,
- harmonic oscillator inward shooting,
- analytic quartic double-well shift,
- larger-box improvement for the double well,
- same-box successive refinement error decrease,
- low-state double-well convergence order,
- parity-specific mismatch scans,
- double-well splitting,
- scattering probability conservation,
- `run_solver` import behavior.

Important latest regression test: `test_square_well_convergence_order()` requires the first square-well states to show near-fourth-order convergence. This prevents the startup-order bug from quietly returning.

8 What the project demonstrates

The project demonstrates three levels of correctness.

8.1 Level 1: Exact benchmarks

The square well and harmonic oscillator have known exact energies. If the numerical method reproduces them, the solver is credible.

Square well:

- validates the basic outward Numerov shooting method,
- checks node structure and parity,
- checks fourth-order convergence after the improved startup values.

Harmonic oscillator:

- validates inward shooting,
- checks parity and node structure,
- checks domain truncation effects,
- supports the Numerov versus RK4 comparison.

8.2 Level 2: Convergence

The project varies:

- grid spacing h,
- domain size $x_{\max}$.

The goal is to show that the energies stabilize. This addresses truncation error, resolution dependence, finite-domain effects, floating-point limits, and root-finding limits. A general lesson in the code is that grid refinement and box truncation should be separated whenever possible. For the quartic double well, h-convergence uses successive refinements at fixed domain size, while $x_{\max}$-convergence compares against a larger-domain reference. For the harmonic-oscillator and RK4 box-size studies, the code instead keeps h approximately fixed while varying $x_{\max}$ so that the curves primarily measure finite-domain truncation.

A key interpretation point is that if a convergence slope is slightly above 4, this does not mean the method is truly higher than fourth order. It usually means the errors are very small and the log-log fit is being affected by numerical floors, root-finding tolerance, or floating-point limits.

8.3 Level 3: New physics

The double well has nearly degenerate even and odd pairs. This shows tunneling splitting, which is the strongest physics result in the project.

The double-well analysis now includes:

- states and densities,
- energy splitting versus parameter b,
- root-finding diagnostics for even and odd parity,
- convergence versus h using successive refinements,
- convergence versus $x_{\max}$ using a larger-box reference,
- an analytic zero shift based on $V_{\min} = -b^2/(4a)$,
- a formulation-aware residual diagnostic: normalized boundary leakage for outward shooting, and final origin-parity mismatch for inward shooting.

9 What is apparent from the lectures and what is beyond them?

9.1 Clearly from the lectures

- ODEs,

- boundary-value problems,
- eigenvalue problems,
- shooting,
- root finding,
- bisection,
- numerical integration,
- numerical differentiation,
- convergence,
- numerical errors,
- documentation.

9.2 Partly from the lectures, but specialized to quantum mechanics

- Numerov method,
- wavefunction normalization,
- parity classification,
- bound-state interpretation,
- inward shooting from a decaying tail,
- tunneling splitting,
- scattering transmission and reflection.

The Numerov idea appears in Pang's Schrödinger equation treatment, but it is more specialized than the main lecture slides.

9.3 Beyond the slides

The most advanced or specialized parts are:

- fourth-order-consistent Numerov startup values,
- startup expansion terms involving $q''(0)$ for the quartic double well,
- high-order derivative stencil at the boundary,
- safeguarded secant polishing after bisection,
- formulation-aware final residual diagnostics, including scale-invariant normalized boundary leakage for outward shooting,
- inward shooting from the forbidden region,
- successive-refinement convergence for non-analytic potentials,
- high-resolution larger-box reference for domain-size studies,
- RK4 versus Numerov method comparison,
- complex Numerov scattering,
- double-barrier resonant tunneling.

9.4 Not used from the lectures

- advection schemes,
- finite-volume methods,
- limiters,
- Burgers equation,
- Euler equations,
- MHD,
- PIC,
- chaos,
- Monte Carlo,
- FFT,
- matrix eigenvalue solvers.

This is fine. The final project explicitly allows choosing a different physics problem or going beyond class material.

10 The clean explanation for a reviewer

Here is the project summary in reviewer language:

> I am solving the one-dimensional time-independent Schrödinger equation as a boundary-value eigenvalue problem. I discretize space on a uniform grid and use the Numerov method to integrate the second-order ODE for a trial energy. Since only special energies satisfy the required boundary conditions, I use a shooting method. The boundary mismatch becomes a scalar function of energy, and I find its roots using bracketing and bisection. For symmetric potentials I use parity to reduce the calculation to the half domain, then reconstruct and normalize the full wavefunction. I validate the solver on the infinite square well and harmonic oscillator, study convergence with grid spacing and domain size, and then apply it to a finite square well and quartic double well to analyze nontrivial bound states and tunneling-induced splitting. For the double well, I separate grid and domain convergence, use an analytic potential shift, and report solver residuals in a formulation-aware way. I also compare Numerov with RK4 for the harmonic oscillator and include scattering through finite and double barriers as a secondary extension.

That is the full logic of the project.